BGS 应用层框架与任务设计 当前工程

日期：2026-02-07
平台：GD32C231G8（48MHz, 64KB Flash） / ADC + DMA / PLS_SO 双边沿 EXTI / NPN 输出
目标：5ms 响应；A/B 相位约 100us，A+B 一周期约 200us

目录

- 0. 概览
- 1. 分层与模块职责
- 2. 采样时序与数据流
- 3. 主循环与中断协作
- 4. 关键任务说明
- 5. param_config.h 结构与寄存器定义
- 6. 参数与掉电保存
- 7. 需要注意的实现点
- 8. 系统总体设计
- 9. 驱动层详细设计
- 10. 应用层详细设计
- 11. 数据结构设计
- 12. 关键算法说明
- 13. 通信与调试

0. 概览

核心检测链路

1. PLS_SO 进入 EXTI5_9（双边沿），仅置位采样标志
2. 主循环触发 MeasureTask，按 PLS_SO 电平选择 A/B DMA 缓冲区
3. Timer2 触发 ADC，DMA 采样 SAMPLE_POINT 点（默认 10）
4. DMA 完成中断置位 g_isSampleDone，进入 AnalyzeTask
5. AnalyzeTask：FastTrimmedMean 后计算 V_A、V_B 与差分 V_OUT
6. OutputTask：与 V_SET 比较得到 presence，驱动 NPN 输出
7. LedTask：Light/Dark 模式显示工作灯

关键参数与信号

- V_SET：阈值（mV），Flash 保存
- led_mode：Light-ON / Dark-ON（Flash 保存）
- PLS_SO：A/B 相位标志，约 10kHz 翻转
- SH_OUT：模拟输出，ADC 采样

> **原则：** presence 是物理检测语义；Light/Dark 只影响输出映射与指示灯显示。

---

1. 分层与模块职责

分层

层级	模块	职责
BSP	bsp_gpio / bsp_adc / bsp_exit / systick	GPIO、ADC+DMA、EXTI、延时
Driver	ISL700_2A	GPIO Bit-bang 写寄存器 + CRC
Service	task_param	参数缓存、NVM A/B 页、CRC32
App	task.c / task_meas / task_analyze / task_output / task_led / task_btn	任务调度、采样、分析、输出、人机

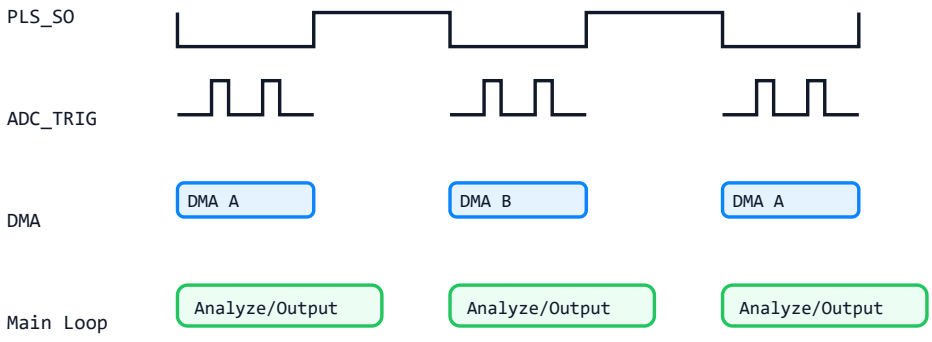
主变量与标志

变量	位置	用途
g_param_cfg	main.c	全局参数缓存（volatile）
g_isStartSampling	main.c	EXTI 触发采样标志
g_isSampleDone	main.c	DMA 完成标志
g_outputState	task_output.c	输出状态机

2. 采样时序与数据流

> Timer2 设定为 100kHz 触发 ADC（48MHz / 480 = 100kHz），SAMPLE_POINT=10，单次采样约 100us。 PLS_SO A/B 相位约 100us，对应一组采样。

流程图（时序）



当前实现要点

- MeasureTask 读取 PLS_SO 电平选择 A/B buffer
- ADC_StartSampling 启动 DMA 采样，完成后由 DMA ISR 置位 g_isSampleDone
- AnalyzeTask 做去极值平均，计算 V_A、V_B 与 V_OUT

```
MeasureTask:
  if PLS_SO == High -> use B buffer
  else              -> use A buffer
  ADC_StartSampling()
```

3. 主循环与中断协作

主循环（superloop）

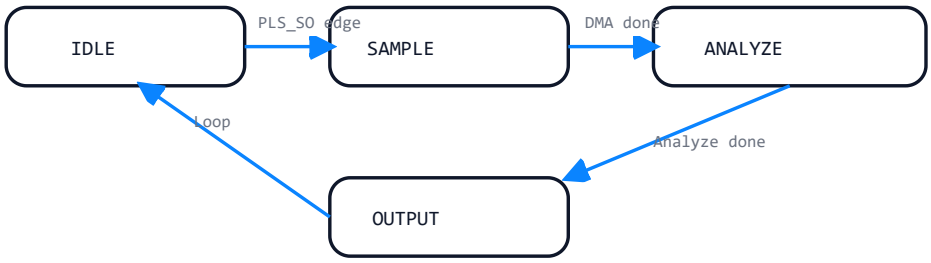
```
while (1) {
  if (g_isSysTickInt) { SysTickTask(); }
  if (g_isStartSampling) { MeasureTask(); }
  if (g_isSampleDone) {
    AnalyzeTask();
    OutputTask();
    LedTask();
  }
}
```

> EXTI 只置位 g_isStartSampling，DMA ISR 只置位 g_isSampleDone；计算工作全部在主循环完成。

中断职责

中断	动作
SysTick	置位 g_isSysTickInt，提供 10ms 按键节拍
EXTI5_9 (PLS_SO)	若上次采样完成，则置位 g_isStartSampling
DMA CH0	采样完成，置位 g_isSampleDone

状态机（采样到输出）



4. 关键任务说明

Task_Init / SysTickTask

文件：task.c

```
void Task_Init(void);
void SysTickTask(void);
```

- Task_Init：ParamCfg_Init → LedTask_Init → BtnTask_Init
- SysTickTask：每 1ms 进入，10ms 调度 BtnTask

MeasureTask / AnalyzeTask

文件：task_meas.c / task_analyze.c

```
void MeasureTask(void);
void AnalyzeTask(void);
uint16_t FastTrimmedMean(const volatile uint16_t *buf,
                         uint16_t len, uint16_t discard);
```

- MeasureTask：按 PLS_SO 电平选择 A/B DMA buffer，并启动 ADC 采样
- AnalyzeTask：计算 V_A、V_B，并得到 V_OUT（当前 V_A - V_B）

OutputTask

文件：task_output.c

```
void OutputTask(void);
extern __IO output_state_t g_outputState;
```

- V_OUT 与 V_SET 比较，得到 OBJECT_PRESENT / NO_OBJECT
- NPN 直接映射 presence（未做 Light/Dark 反相）

BtnTask / LedTask

文件：task_btn.c / task_led.c

```
void BtnTask(void);
void BtnTask_Init(void);
btn_state_t Btn_GetState(void);

void LedTask(void);
void LedTask_Init(void);
void LED_SetMode(led_mode_t mode);
led_mode_t LED_GetMode(void);
```

- 按键去抖后切换 Light/Dark，并调用 ParamCfg_SetLedMode 保存
- 工作灯显示 Light/Dark 模式，电源灯常亮

ParamCfg（参数服务）

文件：task_param.c

```
void ParamCfg_Init(void);
void ParamCfg_ApplyToIC(void);
bool ParamCfg_Save(const volatile param_config_t *param);
bool ParamCfg_SetLedMode(led_mode_t mode);
uint16_t ParamCfg_GetVset(void);
led_mode_t ParamCfg_GetLedMode(void);
void ParamCfg_Print(volatile param_config_t *cfg);
```

- NVM A/B 页 + CRC32，写入成功后更新 g_param_cfg
- ISL700 仅使用 isl700_config_t 部分写寄存器

BSP 驱动接口（关键）

文件：bsp.c / bsp_adc.c / bsp_exit.c / bsp_gpio.c

```
void BSP_Init(void);
void BSP_ICPinConfig(void);
void PLS_SO_InterruptInit(void);
void ADC_UserInit(void);
void ADC_StartSampling(void);
void ADC_SwitchBuffer(uint16_t *buf);
void NPN_Set(void);
void NPN_Clr(void);
```

---

5. param_config.h 结构与寄存器定义

结构体划分

```
typedef struct {
  isl700_config_t isl700;  // 14 bytes
  app_config_t    app;     // 3 bytes
} param_config_t;
```

结构体	字段	用途
isl700_config_t	pls / fine_gain / coarse_gain1 等	IC 寄存器配置（写入 ISL700）
app_config_t	v_set / led_mode	应用层参数（Flash 保存）

ISL700_2A 寄存器字段映射（SR 位段）

SR 位段	字段名	默认值	说明
3:0	fuse_check_low	9	固定不修改
11:4	pls	255	发射脉冲幅值（与 pls-delay 相关）
13:12	no_used	1	保留未使用
23:14	fine_gain	62	细调增益，code 越大增益越大
24	gain_select	0	0=x1, 1=x1.5
28:25	fuse_check_middle	10	固定不修改
32:29	coarse_gain1	15	粗调增益 1
36:33	coarse_gain2	1	粗调增益 2（建议为 0）
40:37	sctrl_res_a	5	前级增益 a（建议 <= 8）
44:41	sctrl_res_b	5	前级增益 b（建议 <= 8）
45	sctrl_internal	1	0=外部电阻，1=内部电阻
49:46	fuse_check_high	6	固定不修改
57:50	fuse_check_floating	0	CRC 计算后由驱动填写

> CRC 由 ISL700_Write_Reg 内部计算并发送，fuse_check_floating 保持默认值即可。

6. 参数与掉电保存

项目	说明
g_param_cfg	全局参数缓存（volatile），ISL700 + APP 参数
ParamCfg_Save	拷贝快照后写入 NVM，写完更新 g_param_cfg
NVM 结构	magic + seq + param + crc32 + padding（双页 A/B）

7. 需要注意的实现点

- 需求为 V = V_B - V_A 且 V <= V_SET 判“检测到目标”，当前实现为 V_A - V_B 且 V_OUT >= V_SET。
- Light/Dark 当前只影响工作灯，不影响 NPN 输出映射。
- main.c 中使用 exti_software_interrupt_enable 进行测试，量产需确认是否移除。

- g_param_cfg 为 volatile，全局读写需注意中断/主循环并发访问。
- 输出响应 5ms，当前采样/分析链路可满足，但需确认滤波窗口与主循环负载。

8. 系统总体设计

软件架构图（驱动层 + 应用层）

```
Application Layer
┌──────────────────────────────────────────────────────────────┐
│ Task_Init / SysTickTask / MeasureTask / AnalyzeTask          │
│ OutputTask / LedTask / BtnTask / ParamCfg                    │
└──────────────────────────────────────────────────────────────┘
Driver/BSP Layer
┌──────────────────────────────────────────────────────────────┐
│ bsp_gpio / bsp_adc / bsp_exit / systick / ISL700_2A          │
└──────────────────────────────────────────────────────────────┘
```

模块依赖关系

- main.c 依赖 BSP_Init / Task_Init / ParamCfg_ApplyToIC
- MeasureTask 依赖 bsp_adc、bsp_exit（PLS_SO）、DMA ISR
- AnalyzeTask 依赖 g_A_Buffer/g_B_Buffer 与参数 V_SET
- OutputTask 依赖 ParamCfg_GetVset 与 NPN GPIO
- BtnTask/LedTask 依赖 ParamCfg_SetLedMode / ParamCfg_GetLedMode

数据流向说明

1. PLS_SO 边沿 → EXTI → g_isStartSampling
2. MeasureTask → ADC + DMA → g_A_Buffer / g_B_Buffer
3. AnalyzeTask → V_A / V_B / V_OUT
4. OutputTask → g_outputState → NPN 输出
5. BtnTask → led_mode → ParamCfg_Save → Flash

硬件平台信息

项目	说明
MCU	GD32C231G8（48MHz, 64KB Flash）
采样接口	ADC 单通道 + DMA，Timer2 触发
相位输入	PLS_SO → EXTI5_9（双边沿）
输出接口	NPN GPIO 输出（bsp_gpio）
调试输出	SEGGER RTT + printf

---

9. 驱动层详细设计

硬件抽象层：GPIO/ADC/EXTI 配置表

接口	用途	文件
GPIO	LED/NPN/BTN/PLS_SO/ISL700 Bit-bang	bsp_gpio.c
ADC + DMA	SH_OUT 采样 + A/B 缓冲区	bsp_adc.c
EXTI	PLS_SO 双边沿中断	bsp_exit.c
SysTick	1ms 时基	systick.c

本工程未使用 I2C/SPI 接口。

引脚映射关系（摘要）

信号	端口	说明
PLS_SO	PA6 (EXTI_6)	相位标志输入
ADC_IN	PA1 (ADC_CH1)	SH_OUT 采样
NPN_OUT	PC14	输出控制
BTN	PA7	按键输入

电气/时序特性（当前实现）

- PLS_SO 翻转：约 10kHz，相位 100us
- ADC 触发：Timer2 100kHz，单次采样 10 点 ≈ 100us
- 响应目标：5ms（当前采样链路可满足）

底层驱动接口映射

需求接口	当前工程对应
sensor_init()	BSP_Init() + ParamCfg_Init() + ParamCfg_ApplyToIC()
sensor_read_status()	AnalyzeTask() / g_outputState
sensor_configure()	ParamCfg_SetIC() / ParamCfg_SetApp()
sensor_isr_handler()	EXTI5_9_IRQHandler() / DMA_Channel0_IRQHandler()
sensor_error_handle()	当前为占位（无统一错误处理）

中断服务流程

- 触发条件：PLS_SO 边沿 / DMA 传输完成 / SysTick 1ms
- 临界区：ISR 只置位标志，不做复杂计算
- 标志位：g_isStartSampling / g_isSampleDone / g_isSysTickInt
- 事件传递：主循环轮询标志，调用任务处理

硬件初始化序列（按时间顺序）

1. SysTick 初始化（1ms 时基）
2. GPIO 初始化（LED/NPN/BTN/PLS_SO/ISL700 IO）
3. PLS_SO EXTI 配置（双边沿）
4. ADC + DMA + Timer2 配置
5. ParamCfg_Init 读取 NVM 默认参数
6. ParamCfg_ApplyToIC 配置 ISL700_2A

---

10. 应用层详细设计

API 接口规范（当前工程）

函数	参数	返回值	调用方式
MeasureTask()	无	void	异步（由 EXTI 标志驱动）
AnalyzeTask()	无	void	同步（主循环）
OutputTask()	无	void	同步（主循环）
ParamCfg_Save()	param	bool	同步（调用方等待）

当前工程未使用回调机制，采用标志位 + 主循环调度。

业务逻辑层状态机

- 空闲 → 检测中：PLS_SO 边沿触发 g_isStartSampling
- 检测中 → 触发：DMA 完成后 AnalyzeTask 计算完成
- 触发 → 空闲：完成 OutputTask / LedTask 后回到等待
- 错误恢复：当前为占位，未实现统一错误恢复策略

阈值管理逻辑

- 阈值 V_SET 存放在 g_param_cfg.app.v_set
- OutputTask 直接比较 V_OUT 与 V_SET
- 动态修改通过 ParamCfg_SetVset / ParamCfg_Save

多传感器协同

当前工程为单通道单传感器设计，无多传感器协同机制。

配置管理方案

- 默认参数：PARAM_CONFIG_DEFAULT
- 运行时修改：ParamCfg_SetApp / ParamCfg_SetIC / ParamCfg_Save
- 掉电保存：NVM A/B 页 + CRC32
- 合法性检查：目前为隐式（未做范围校验）

---

11. 数据结构设计

```
typedef struct {
  uint16_t v_set;
  uint8_t  led_mode;
} app_config_t;

typedef struct {
  isl700_config_t isl700;
  app_config_t    app;
} param_config_t;

typedef enum {
  NO_OBJECT = 0,
  OBJECT_PRESENT = 1,
  SHORT_CIRCUIT = -1
} output_state_t;
```

需求模板中的 SensorConfig/SensorState 可映射为 app_config_t 与 output_state_t。

---

12. 关键算法说明

数字滤波算法

- FastTrimmedMean：对采样点去极值后平均
- 异常值剔除：discard N（默认 1）
- 未实现移动平均与输出去抖

校准算法

- 当前无自动基线校准
- 无环境光补偿与温漂补偿

---

13. 通信与调试

与主控的数据交换

当前工程无外部通信协议（未集成 UART/I2C/SPI 数据交换）。

调试输出格式

```
printf("V_A: %u mV\r\n", V_A);
printf("V_B: %u mV\r\n", V_B);
printf("Object Present!\r\n");
```

底层使用 SEGGER RTT 进行 printf 输出。

本说明基于当前工程源码梳理，用于功能对齐与架构沟通。